Practical 5 - Nonlinear equations

Xiru ZHANG

Year 2023/2024

Contents

1 Calculation of roots of nonlinear functions **2**
1.1 Implementation of Newton'smethod 2
1.2 The approximate solution of $x^3 = \cos(x)$ 2

2 Newton's p-adic method **2**
2.1 Newton's p-adic method 2
2.2 Test 3

3 Inversion of a matrix **3**
3.1 Verification 3
3.2 Program 4

1 Calculation of roots of nonlinear functions

1.1 Implementation of Newton'smethod

Listing 1: Newton'smethod of nonlinear functions Implementation

```
function [root, iterations, convergence] = newtonMethod(
   f, df, x0, tol, max_iter)
iterations = 0;
convergence = [];
x = x0;
while true
    fx = f(x);
    dfx = df(x);
    if dfx == 0
        error('Zero␣derivative.␣No␣solution␣found.');
    end
    x_new = x - fx/dfx;
    convergence(end+1) = abs(x_new - x);
    x = x_new;
    iterations = iterations + 1;
    if (abs(fx) < tol) || (iterations >= max_iter)
        break;
    end
end
root = x;
end
```

1.2 The approximate solution of $x^3 = \cos(x)$

The application of Newton's method for solving the transcendental equation $x^3 = \cos(x)$ yielded a numerical solution. After 6 iterations, the method converged to a root at approximately $x = 0.865474$. The convergence process, characterized by the absolute differences between successive approximations, demonstrated a rapid decline in error with values: 0.6121, 0.2025, 0.0424, 0.0018, following down to 0.0000 for the last two iterations, indicating the convergence criteria were satisfied.

2 Newton's p-adic method

2.1 Newton's p-adic method

Listing 2: Newton's p-adic method Implementation

```
function x_k = newtonPadic(f, df, p, k, x1)
    x_k = x1;
```

```
    for i = 1:k
        f_val = mod(f(x_k), p^i);
        df_val = mod(df(x_k), p^i);
        if mod(df_val, p) == 0
            x_k = NaN;
            return;
        end
        df_inv = modinv(df_val, p^i);
        x_k = mod(x_k - df_inv * f_val, p^i);
    end
end

function inv = modinv(a, m)
    [g, inv, ~] = gcd(a, m);
    if g ~= 1
        inv = NaN;
    else
        inv = mod(inv, m);
    end
end
```

2.2 Test

Applying the modified p-adic Newton's method to the polynomial $f(x) = x^2 - 2$ with a prime $p = 7$ and initial guess $x_1 = 3$, we seek solutions modulo 7^k for $k = 2, 3, 4$. The obtained results are as follows:

- For $k = 2$, corresponding to modulo 49, the solution is $x = 10$.
- For $k = 3$, corresponding to modulo 343, the solution is $x = 108$.
- For $k = 4$, corresponding to modulo 2401, the solution is $x = 2166$.

These results indicate that the iterative method successfully computes a sequence of approximations that converge to a root of $f(x)$ modulo p^k for increasing values of k. Each solution x satisfies the congruence $f(x) \equiv 0 \mod p^k$, validating the efficacy of the p-adic Newton's method for these parameters. It is important to note that for higher powers of p, the values of x tend to increase, as seen in the transition from 10 to 108 to 2166, which is expected due to the nature of p-adic numbers and the increasing complexity of the congruences.

3 Inversion of a matrix

3.1 Verification

To show that Newton's iteration for matrix inversion verifies the relation $X_{n+1} = 2X_n - X_nAX_n$, we start with the function $f(X) = A - X^{-1}$. The goal is to find

a matrix X such that $f(X) = 0$, which implies that $X = A^{-1}$.

Newton's method for finding a root (zero) of a function can be adapted for matrices. The general update rule for Newton's method is:

$$X_{n+1} = X_n - \left[\frac{df}{dX}(X_n)\right]^{-1} f(X_n)$$

For the function $f(X)$, the derivative with respect to X, denoted as $\frac{df}{dX}$, is the matrix derivative of f concerning X. Since $f(X) = A - X^{-1}$, we differentiate $f(X)$ with respect to X treating X as a variable and A as a constant matrix, which yields $\frac{df}{dX} = X^{-2}$. Thus, the Newton's method update rule for this function becomes:

$$\begin{aligned}
X_{n+1} &= X_n - \left(X_n^{-2}\right)^{-1}(A - X_n^{-1}) \\
X_{n+1} &= X_n - X_n^2(A - X_n^{-1}) \\
X_{n+1} &= X_n - X_n(AX_n - I) \\
X_{n+1} &= X_n - X_nAX_n + X_n \\
X_{n+1} &= 2X_n - X_nAX_n
\end{aligned}$$

This is the iterative step of Newton's method for matrix inversion.

3.2 Program

Listing 3: Inversion of matrix Implementation

```
function X_inv = invertMatrixUsingNewtonMethod(A)
    n = size(A, 1);
    X = (A' / trace(A' * A));
    max_iter = 1000;
    tol = 1e-8;
    for i = 1:max_iter
        X_new = 2*X - X*(A*X);
        err = norm(eye(n) - X_new*A, 2);
        if err < tol
            break;
        end
        X = X_new;
    end
    X_inv = X;
end
```